

CS501

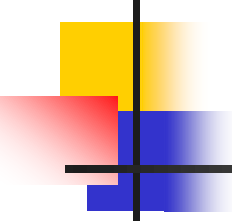
Introductory Computation for Scientists



Dr. Ash Pahwa

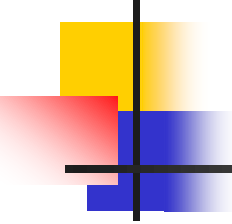
Lesson 4:

The Core Python Language - 2



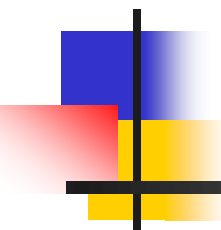
Outline

- Collections
 - Lists
 - Tuples
 - Dictionary
- Iteration (Loops)
 - For and While loops
 - Comparisons – If-elseif-else
- String Formatting
- File IO

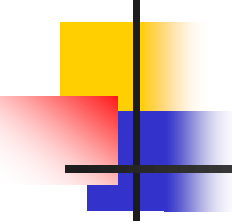


Python Data Structures

	Package • Data Structure	Create	Index Start=0	Slice	Edit
1	Core Python • String	s='abcdefghi'	s[2]	s[start:end:inc] s[2:8:2]	Immutable
2	Core Python • List	1D: l = [1,2,3] 2D: l = [1,2,list1]	1D: l[2] 2D: l[2][3]	l[start:end:inc] l[2:8:2]	Mutable
3	Core Python • Tuples	1D: t = (1,2,3) 2D: t = (1,2,list)	1D: t[2] 2D: t[2][3]	t[start:end:inc] t[2:8:2]	Immutable
4	Core Python • Dictionary	1D: d = {1:1, 2:2, 3:3 } 2D: d = {1:1, 2:2, 3:list }	Indexing Using keys		Mutable
5	Numpy • Arrays	np.array(List)	1D: a[2] 2D: a[2][3]	a[start:end:inc] a[2:8:2]	Mutable
6	Pandas • Series • DataFrame	1D: pd.Series(<index>,<data>) 2D: pd.DataFrame(<s1>,<s2>, ..)	s.loc[<index>] s.iloc[2] df.loc['index']['c1','c2']		Mutable
7	Datascience • Table	2D: t1 = Table().with_column('col name', <data>, ...)	2D: t1.select('col name')	2D: t1.where('con dition')	Mutable



Lists



Create a List

1. Lists

```
In [1]: list1 = [1, 'two', 3.14, 0]  
list1
```

```
Out[1]: [1, 'two', 3.14, 0]
```

List of a List

1. Lists

```
In [1]: list1 = [1, 'two', 3.14, 0]
        list1
Out[1]: [1, 'two', 3.14, 0]
```

1.1 List of a list

```
In [2]: a = 4
        list2 = [2, a, -0.1, list1, True]
        list2
Out[2]: [2, 4, -0.1, [1, 'two', 3.14, 0], True]

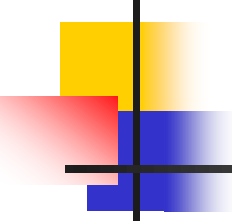
In [3]: list1[2]
Out[3]: 3.14

In [4]: list2[-1]
Out[4]: True

In [5]: list2[3][1]
Out[5]: 'two'

In [6]: list2[3][1][1]
Out[6]: 'w'
```

list2		0	1	2	3
0		2			
1		4			
2		-0.1			
3	List1	1	'two'	3.14	0
4		True			



Membership

1. Lists

```
In [1]: list1 = [1, 'two', 3.14, 0]
        list1
Out[1]: [1, 'two', 3.14, 0]
```

1.1 List of a list

```
In [2]: a = 4
        list2 = [2, a, -0.1, list1, True]
        list2
Out[2]: [2, 4, -0.1, [1, 'two', 3.14, 0], True]
```

1.2 Membership in a list

```
In [7]: 1 in list1
Out[7]: True

In [8]: 'two' in list2
Out[8]: False
```

Lists (unlike Strings) are Mutable

Lists and Mutability

```
In [10]: list1
```

```
Out[10]: [1, 'two', 3.14, 0]
```

```
In [11]: list1[2] = 2.72
```

```
In [12]: list1
```

```
Out[12]: [1, 'two', 2.72, 0]
```

```
In [13]: q1 = [1,2,3]
```

```
In [14]: q2 = q1
```

```
In [15]: q1[2] = 'oops'
```

```
In [16]: q1
```

```
Out[16]: [1, 2, 'oops']
```

```
In [17]: q2
```

```
Out[17]: [1, 2, 'oops']
```

Slicing with Lists

Slicing similar to string

```
In [18]: q1 = [0., 0.1, 0.2, 0.3, 0.4, 0.5]  
         q1[1:4]
```

```
Out[18]: [0.1, 0.2, 0.3]
```

```
In [19]: q1[::-1]
```

```
Out[19]: [0.5, 0.4, 0.3, 0.2, 0.1, 0.0]
```

```
In [20]: q1[1::2]
```

```
Out[20]: [0.1, 0.3, 0.5]
```

```
In [21]: q2 = q1[1:4]
```

```
In [22]: q2
```

```
Out[22]: [0.1, 0.2, 0.3]
```

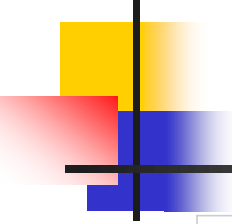
```
In [23]: q2[1] = 99
```

```
In [24]: q2
```

```
Out[24]: [0.1, 99, 0.3]
```

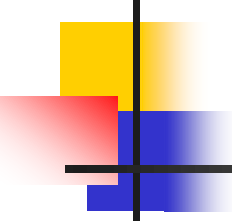
```
In [25]: q1
```

```
Out[25]: [0.0, 0.1, 0.2, 0.3, 0.4, 0.5]
```



List Methods

Method	Description
<u>append()</u>	Adds an element at the end of the list
<u>clear()</u>	Removes all the elements from the list
<u>copy()</u>	Returns a copy of the list
<u>count()</u>	Returns the number of elements with the specified value
<u>extend()</u>	Add the elements of a list (or any iterable), to the end of the current list
<u>index()</u>	Returns the index of the first element with the specified value
<u>insert()</u>	Adds an element at the specified position
<u>pop()</u>	Removes the element at the specified position
<u>remove()</u>	Removes the first item with the specified value
<u>reverse()</u>	Reverses the order of the list
<u>sort()</u>	Sorts the list



List Methods

1.5 List Methods

```
In [25]: q=[]
```

```
In [26]: q.append(4)
```

```
In [27]: q
```

```
Out[27]: [4]
```

```
In [28]: q.extend([6,7,8])
```

```
In [29]: q
```

```
Out[29]: [4, 6, 7, 8]
```

```
In [30]: q.insert(1,5)
```

```
In [31]: q
```

```
Out[31]: [4, 5, 6, 7, 8]
```

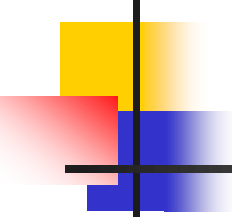
```
In [32]: q.remove(7)
```

```
In [33]: q
```

```
Out[33]: [4, 5, 6, 8]
```

```
In [34]: q.index(8)
```

```
Out[34]: 3
```



Sorting of Lists

- 'Sort' is a method of list
 - Sorts a list
- 'Sorted' is a function
 - That returns a sorted List
 - But the original list remains the same

▼ 1.6 Sort and Reverse

```
[36] q = [2,0,4,3,1]
```

```
[37] q.sort()  
q
```

```
↳ [0, 1, 2, 3, 4]
```

```
▶ q.reverse()  
q
```

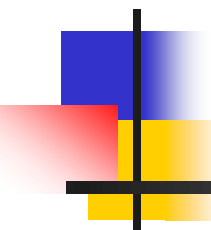
```
↳ [4, 3, 2, 1, 0]
```

```
[39] sorted(q)
```

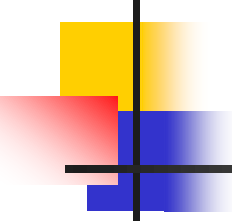
```
↳ [0, 1, 2, 3, 4]
```

```
▶ q
```

```
↳ [4, 3, 2, 1, 0]
```



Tuples



Tuple Creation

Tuples

```
In [2]: t = (1, 'two', 3.)
```

```
In [3]: t
```

```
Out[3]: (1, 'two', 3.0)
```

Tuples Indexing and Slicing

Tuples indexing and slicing

```
In [4]: t[1]
```

```
Out[4]: 'two'
```

```
In [5]: t[2]
```

```
Out[5]: 3.0
```

```
In [6]: t[2] = 4
```

```
-----  
TypeError                                 Traceback (most recent call last)  
<ipython-input-6-56c309f1aa91> in <module>()  
----> 1 t[2] = 4
```

```
TypeError: 'tuple' object does not support item assignment
```

```
In [7]: t = (1, ['a', 'b', 'd'], 0)
```

```
In [8]: t[1][2] = 'c'
```

```
In [9]: t
```

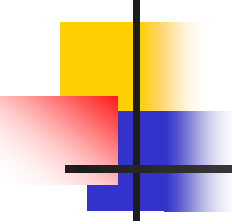
```
Out[9]: (1, ['a', 'b', 'c'], 0)
```

Tuples

```
[2]: t = (1, 'two', 3.)
```

```
[3]: t
```

```
[3]: (1, 'two', 3.0)
```



Tuple Packing

Tuple Packing

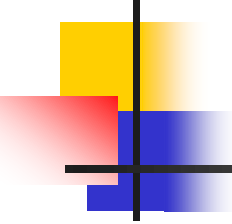
```
In [10]: a,b,c = 97,98,99
```

```
In [11]: b
```

```
Out[11]: 98
```

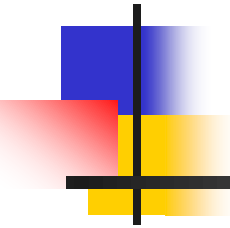
```
In [12]: c
```

```
Out[12]: 99
```

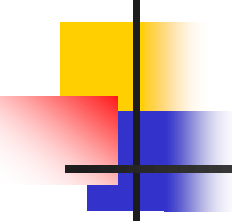


Tuple Methods

Method	Description
<u>count()</u>	Returns the number of times a specified value occurs in a tuple
<u>index()</u>	Searches the tuple for a specified value and returns the position of where it was found

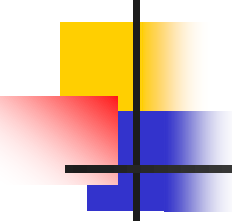


Dictionaries



Dictionaries

- Python dictionary is an unordered collection of items. While other compound data types have only value as an element, a dictionary has a key: value pair.
- Dictionaries are optimized to retrieve values when the key is known.



Dictionary

3. Dictionary

```
In [44]: # empty dictionary  
my_dict1 = {}  
my_dict1
```

```
Out[44]: {}
```

```
In [45]: # dictionary with integer keys  
my_dict2 = {1: 'apple', 2: 'ball'}  
my_dict2
```

```
Out[45]: {1: 'apple', 2: 'ball'}
```

```
In [46]: # dictionary with mixed keys  
my_dict3 = {'name': 'John', 1: [2, 4, 3]}  
my_dict3
```

```
Out[46]: {'name': 'John', 1: [2, 4, 3]}
```

```
In [47]: # using dict()  
my_dict4 = dict({1: 'apple', 2: 'ball'})  
my_dict4
```

```
Out[47]: {1: 'apple', 2: 'ball'}
```

```
In [48]: # from sequence having each item as a pair  
my_dict5 = dict([(1, 'apple'), (2, 'ball')])  
my_dict5
```

```
Out[48]: {1: 'apple', 2: 'ball'}
```

Access Elements from a Dictionary

3.1 Access Elements from a Dictionary

```
In [49]: my_dict = {'name': 'Jack', 'age': 26}
```

```
# Output: Jack
print(my_dict['name'])

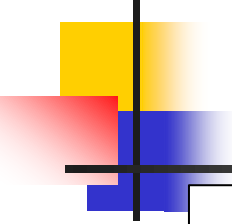
# Output: 26
print(my_dict.get('age'))
```

```
Jack
26
```

```
In [50]: # Trying to access keys which doesn't exist throws error
my_dict.get('address')
my_dict['address']
```

```
-----
KeyError                                Traceback (most recent call last)
<ipython-input-50-8d0b31e209ba> in <module>()
      1 # Trying to access keys which doesn't exist throws error
      2 my_dict.get('address')
----> 3 my_dict['address']

KeyError: 'address'
```



Data Modification in Dictionary

3.2 Add or Change Elements in a Dictionary

```
In [54]: my_dict = {'name': 'Jack', 'age': 26}
          print(my_dict)

          # update value
          my_dict['age'] = 27
          print(my_dict)

          # add item
          my_dict['address'] = 'Downtown'
          print(my_dict)

{'name': 'Jack', 'age': 26}
{'name': 'Jack', 'age': 27}
{'name': 'Jack', 'age': 27, 'address': 'Downtown'}
```

Delete From Dictionary

3.3 Delete Elements from a Dictionary

```
In [55]: # create a dictionary
squares = {1:1, 2:4, 3:9, 4:16, 5:25}

# remove a particular item
# Output: 16
print(squares.pop(4))

16
```

```
In [56]: # Output: {1: 1, 2: 4, 3: 9, 5: 25}
print(squares)

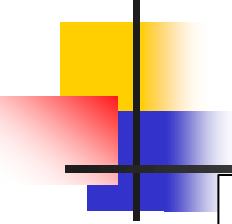
{1: 1, 2: 4, 3: 9, 5: 25}
```

```
In [57]: # remove an arbitrary item
# Output: (1, 1)
print(squares.popitem())

(5, 25)
```

```
In [58]: # Output: {2: 4, 3: 9, 5: 25}
print(squares)

{1: 1, 2: 4, 3: 9}
```



Delete From Dictionary

```
In [62]: # delete a particular item
squares = {1:1, 2:4, 3:9, 4:16, 5:25}
del squares[5]
print(squares)
```

```
{1: 1, 2: 4, 3: 9, 4: 16}
```

```
In [63]: # remove all items
squares.clear()
print(squares)
```

```
{}
```

```
In [64]: # delete the dictionary itself
del squares
print(squares)
```

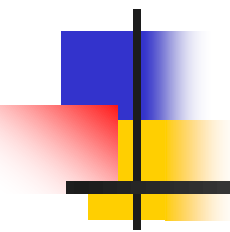
```
-----
NameError                                Traceback (most recent call last)
<ipython-input-64-b13991604d5d> in <module>()
      2 del squares
      3
----> 4 print(squares)

NameError: name 'squares' is not defined
```



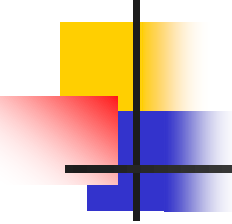
Dictionary Methods

Method	Description
<u>clear()</u>	Removes all the elements from the dictionary
<u>copy()</u>	Returns a copy of the dictionary
<u>fromkeys()</u>	Returns a dictionary with the specified keys and value
<u>get()</u>	Returns the value of the specified key
<u>items()</u>	Returns a list containing a tuple for each key value pair
<u>keys()</u>	Returns a list containing the dictionary's keys
<u>pop()</u>	Removes the element with the specified key
<u>popitem()</u>	Removes the last inserted key-value pair
<u>setdefault()</u>	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
<u>update()</u>	Updates the dictionary with the specified key-value pairs
<u>values()</u>	Returns a list of all the values in the dictionary



Looping

For and While Loops



For Loop

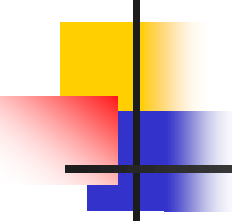
Loops

```
In [13]: fruit_list = ['apple', 'melon', 'banana', 'orange']  
for fruit in fruit_list:  
    print(fruit)
```

```
apple  
melon  
banana  
orange
```

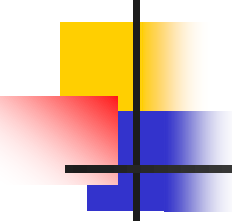
```
In [14]: fruit_list = ['apple', 'melon', 'banana', 'orange']  
for fruit in fruit_list:  
    for letter in fruit:  
        print(letter, end='.')  
    print()
```

```
a.p.p.l.e.  
m.e.l.o.n.  
b.a.n.a.n.a.  
o.r.a.n.g.e.
```



Difference between 'range' and 'arange'

- The main difference between the two is that
 - range is a built-in Python class,
 - while arange() is a function that belongs to a third-party library (NumPy)
- Their purposes are different!
 - Generally, range is more suitable when you need to iterate using the Python for loop



Difference between 'range' and 'arange'

```
In [4]: for i in range(1,10,2):  
        print(i)
```

```
1  
3  
5  
7  
9
```

```
In [5]: import numpy as np  
        for i in np.arange(1,10,2):  
            print(i)
```

```
1  
3  
5  
7  
9
```

Difference between 'range' and 'arange'

```
In [20]: a = range(5)  
         print(a)
```

```
range(0, 5)
```

```
In [22]: b = range(1,6)  
         b
```

```
Out[22]: range(1, 6)
```

```
In [25]: c = list(range(0,6))  
         c
```

```
Out[25]: [0, 1, 2, 3, 4, 5]
```

```
In [28]: type(c)
```

```
Out[28]: list
```

```
In [26]: import numpy as np  
         d = np.arange(0,10,2)  
         d
```

```
Out[26]: array([0, 2, 4, 6, 8])
```

```
In [29]: type(d)
```

```
Out[29]: numpy.ndarray
```

Fibonacci Numbers Using 'range' and 'arange'

```
In [17]: n = 30
fib = [1,1]
for i in range(2, n+1):
    fib.append(fib[i-1] + fib[i-2])

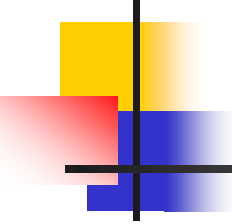
print(fib)
```

```
[1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, 17711, 28657, 46368, 75025, 121393, 196418, 317811, 514229, 832040, 1346269]
```

```
In [31]: n = 30
fib = [1,1]
for i in np.arange(2, n+1):
    fib.append(fib[i-1] + fib[i-2])

print(fib)
```

```
[1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, 17711, 28657, 46368, 75025, 121393, 196418, 317811, 514229, 832040, 1346269]
```

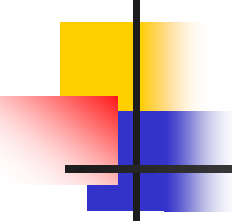


While Loop

While Loop

```
In [32]: i = 0
         while i < 10:
             i += 1
             print(i, end='.')
```

```
print()
1.2.3.4.5.6.7.8.9.10.
```

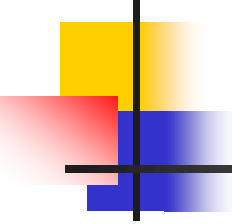


While with a 'break'

True	False
Any Number	0

```
In [33]: x = 0
while True:
    x += 1
    if not (x % 15 or x % 25):
        break

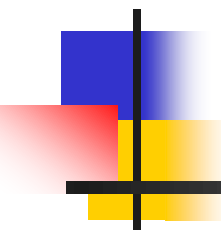
print(x, 'is divisible by both 15 and 25')
75 is divisible by both 15 and 25
```



If ... elseif ... else

```
In [37]: a = 35
         b = 34
         if b > a:
             print("b is greater than a")
         elif a == b:
             print("a and b are equal")
         else:
             print("a is greater than b")

a is greater than b
```



List Comprehension

List Comprehension

Ex# 1 + 2 + 3

```
[9] # Examples of List Comprehension  
#  
# Example#1
```

```
list1 = []  
for item in range(1,6):  
    list1.append(item)
```

```
list1
```

```
[1, 2, 3, 4, 5]
```

```
[10] # Example#2
```

```
list2 = [item for item in range(1,6)]
```

```
list2
```

```
[1, 2, 3, 4, 5]
```

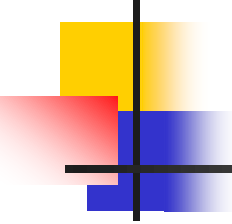


```
# Example#3
```

```
list3 = [item ** 3 for item in range(1,6)]
```

```
list3
```

```
[1, 8, 27, 64, 125]
```



Example#4:

List Comprehension With an IF Statement



```
# Example#4 : List Comprehension with If Clauses
```

```
list4 = [item for item in range(1,11) if item %2 == 0]
```

```
list4
```

```
☞ [2, 4, 6, 8, 10]
```

Example#5:

List Comprehension that processes Another List's elements

```
# Example#5 : List Comprehension that processes Another List's elements
```

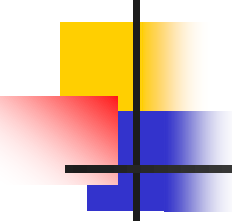
```
colors = ['red', 'orange', 'yellow', 'green', 'blue']
```

```
colors2 = [item.upper() for item in colors]
```

```
print(colors2)
```

```
print(colors)
```

```
['RED', 'ORANGE', 'YELLOW', 'GREEN', 'BLUE']  
['red', 'orange', 'yellow', 'green', 'blue']
```



Example#6:

List Comprehension Creating Tuples

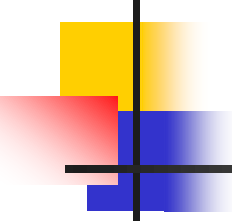


```
# Example#6 : List Comprehension Creating Tuples
```

```
cubes = [(x, x**3) for x in range(1,6)]
```

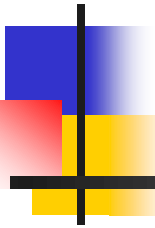
```
cubes
```

```
[(1, 1), (2, 8), (3, 27), (4, 64), (5, 125)]
```

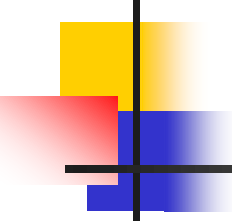


Example#7: Multiples

```
▶ # Example#7 : Multiples  
multiples = [x for x in range(3,30,3)]  
multiples  
[3, 6, 9, 12, 15, 18, 21, 24, 27]
```

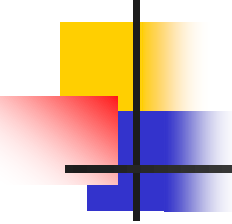


Anonymous Function: lambda



Lambda Function

- Formal method of defining a function
- `def <function_name> (<parameter_list>):`
 - Return expression
- -----
- Anonymous Function
- `lambda <parameter_list>: <expression>`



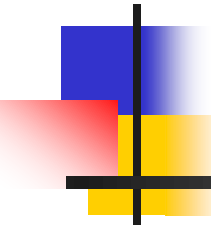
Example#8: Lambda Function

```
[22] # Example#8: Lambda Function  
  
# Find all the odd numbers  
  
numbers = [ 10, 3, 7, 1, 9, 4, 2, 8, 5, 6]  
  
list(filter(lambda x: x%2 != 0, numbers))  
  
[3, 7, 1, 9, 5]
```

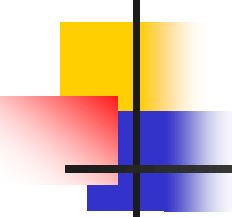


```
list(map(lambda x: x ** 2, numbers))
```

```
[100, 9, 49, 1, 81, 16, 4, 64, 25, 36]
```



String Formatting



String Formatting

String Formatting

```
In [2]: '{} plus {} equals {}'.format(2,3,'five')
```

```
Out[2]: '2 plus 3 equals five'
```

```
In [3]: '{1} plus {0} equals {2}'.format(2,3,'five')
```

```
Out[3]: '3 plus 2 equals five'
```

```
In [4]: '{num1} plus {num2} equals {answer}'.format(num1=2, num2=3, answer='five')
```

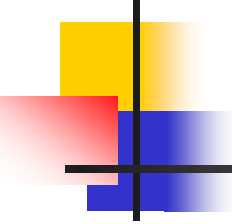
```
Out[4]: '2 plus 3 equals five'
```

```
In [5]: '{0} plus {0} equals {1}'.format(2, 2+2)
```

```
Out[5]: '2 plus 2 equals 4'
```

```
In [6]: '=== {0:12} ==='.format('Python')
```

```
Out[6]: '=== Python           ==='
```



String Formatting

Decimal, Binary, Octal, Hex

```
[8] a = 254  
    'a = {0:5d}'.format(a)
```

```
'a =    254'
```

```
[9] # Print the number in Decial  
    a = 254  
    print('a = {0:5d}'.format(a))
```

```
a =    254
```

```
[10] # Print the number in Binary  
      print('a = {0:10b}'.format(a))
```

```
a =    11111110
```

```
▶ # Print the number in Octal  
  print('a = {0:5o}'.format(a))
```

```
↳ a =    376
```

```
[12] # Print the number in Hex  
      print('a = {0:5x}'.format(a))
```

```
a =     fe
```

```
▶ # Print the number in Hex  
    'a = {0:5X}'.format(a)
```

```
'a =     FE'
```

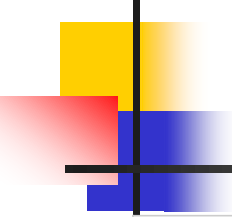
String Formatting Floating Point Numbers

```
▶ x = 34.567898
print('x = {:.2f} 2 places after the decimal point'.format(x))
print('x = {:.3f} 3 places after the decimal point'.format(x))
print('x = {:.4f} 4 places after the decimal point'.format(x))
print('x = {:.5f} 5 places after the decimal point'.format(x))
print('x = {:.6f} 6 places after the decimal point'.format(x))
print('x = {:.7f} 7 places after the decimal point'.format(x))
```

```
↳ x = 34.57 2 places after the decimal point
x = 34.568 3 places after the decimal point
x = 34.5679 4 places after the decimal point
x = 34.56790 5 places after the decimal point
x = 34.567898 6 places after the decimal point
x = 34.5678980 7 places after the decimal point
```

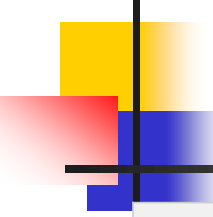
```
▶ y = 956
print('x = {:.7f} 7 places after the decimal point & {:3d} number with 3 digits'.format(x,y))

x = 34.5678980 7 places after the decimal point & 956 number with 3 digits
```



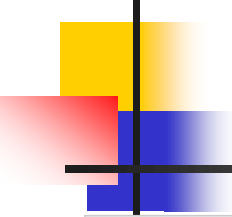
String Methods - 1

Method	Description
<u>capitalize()</u>	Converts the first character to upper case
<u>casefold()</u>	Converts string into lower case
<u>center()</u>	Returns a centered string
<u>count()</u>	Returns the number of times a specified value occurs in a string
<u>encode()</u>	Returns an encoded version of the string
<u>endswith()</u>	Returns true if the string ends with the specified value
<u>expandtabs()</u>	Sets the tab size of the string
<u>find()</u>	Searches the string for a specified value and returns the position of where it was found
<u>format()</u>	Formats specified values in a string
<u>format_map()</u>	Formats specified values in a string



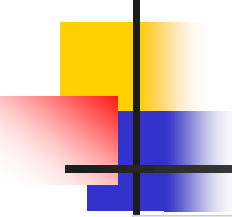
String Methods - 2

<u>index()</u>	Searches the string for a specified value and returns the position of where it was found
<u>isalnum()</u>	Returns True if all characters in the string are alphanumeric
<u>isalpha()</u>	Returns True if all characters in the string are in the alphabet
<u>isdecimal()</u>	Returns True if all characters in the string are decimals
<u>isdigit()</u>	Returns True if all characters in the string are digits
<u>isidentifier()</u>	Returns True if the string is an identifier
<u>islower()</u>	Returns True if all characters in the string are lower case
<u>isnumeric()</u>	Returns True if all characters in the string are numeric
<u>isprintable()</u>	Returns True if all characters in the string are printable
<u>isspace()</u>	Returns True if all characters in the string are whitespaces



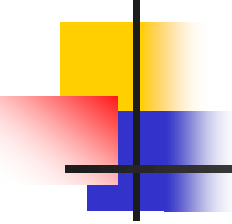
String Methods - 3

<u>istitle()</u>	Returns True if the string follows the rules of a title
<u>isupper()</u>	Returns True if all characters in the string are upper case
<u>join()</u>	Joins the elements of an iterable to the end of the string
<u>ljust()</u>	Returns a left justified version of the string
<u>lower()</u>	Converts a string into lower case
<u>lstrip()</u>	Returns a left trim version of the string
<u>maketrans()</u>	Returns a translation table to be used in translations
<u>partition()</u>	Returns a tuple where the string is parted into three parts
<u>replace()</u>	Returns a string where a specified value is replaced with a specified value
<u>rfind()</u>	Searches the string for a specified value and returns the last position of where it was found
<u>rindex()</u>	Searches the string for a specified value and returns the last position of where it was found



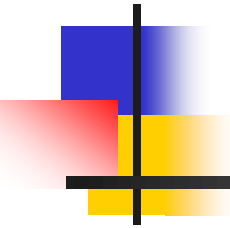
String Methods - 4

<u>rjust()</u>	Returns a right justified version of the string
<u>rpartition()</u>	Returns a tuple where the string is parted into three parts
<u>rsplit()</u>	Splits the string at the specified separator, and returns a list
<u>rstrip()</u>	Returns a right trim version of the string
<u>split()</u>	Splits the string at the specified separator, and returns a list
<u>splitlines()</u>	Splits the string at line breaks and returns a list
<u>startswith()</u>	Returns true if the string starts with the specified value
<u>strip()</u>	Returns a trimmed version of the string
<u>swapcase()</u>	Swaps cases, lower case becomes upper case and vice versa
<u>title()</u>	Converts the first character of each word to upper case

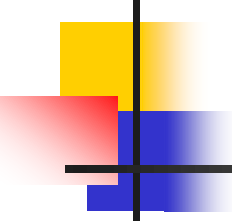


String Methods - 5

<u>translate()</u>	Returns a translated string
<u>upper()</u>	Converts a string into upper case
<u>zfill()</u>	Fills the string with a specified number of 0 values at the beginning

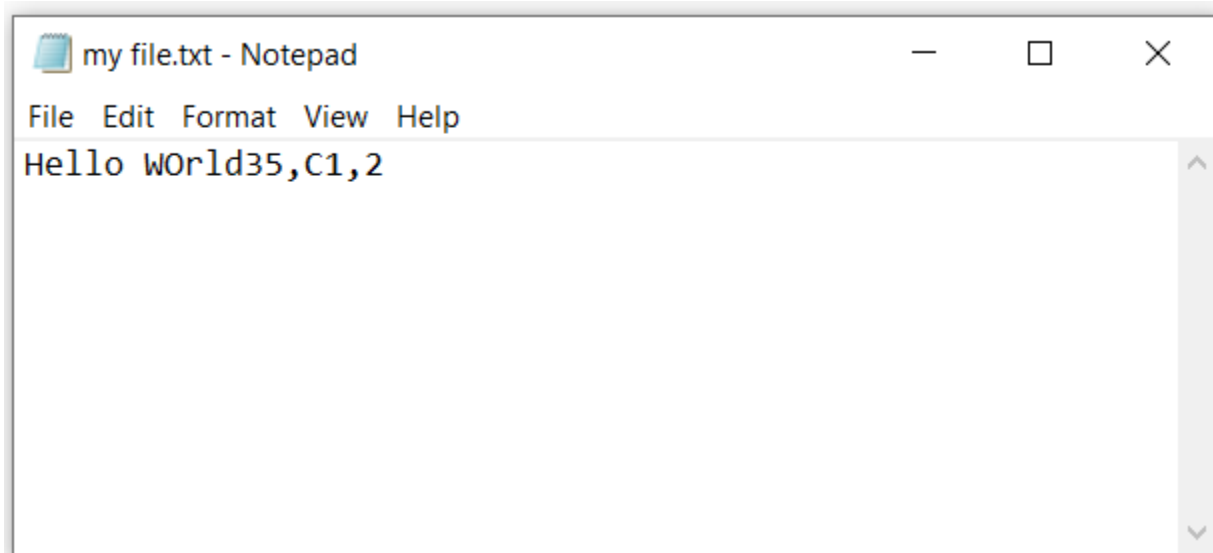


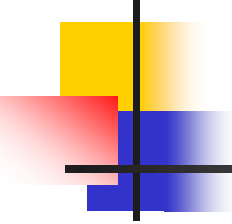
File Input / Output



File IO

```
In [10]: f = open('my file.txt','w')  
  
In [11]: f.write("Hello wOrld")  
Out[11]: 11  
  
In [12]: print(35, 'C1', 2, sep=',', file=f)  
  
In [13]: f.close()
```



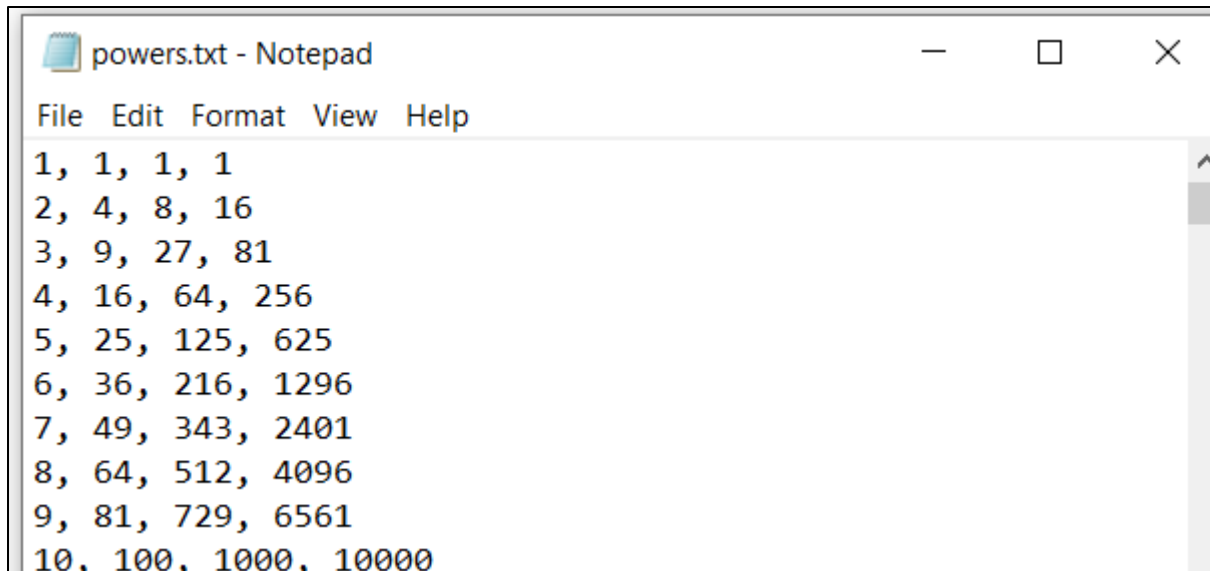


File IO

```
In [22]: f = open('powers.txt','w')
         for i in range(1,1001):
             print(i, i**2, i**3, i**4, sep=', ', file=f)
         f.close()
```

```
In [23]: f = open('powers.txt', 'r')
         squares, cubes, fourths = [], [], []
         for line in f.readlines():
             fields = line.split(',')
             squares.append(int(fields[1]))
             cubes.append(int(fields[2]))
             fourths.append(int(fields[3]))
         f.close()

         n = 500
         print(n, 'cubed is', cubes[n-1])
```



```
powers.txt - Notepad
File Edit Format View Help
1, 1, 1, 1
2, 4, 8, 16
3, 9, 27, 81
4, 16, 64, 256
5, 25, 125, 625
6, 36, 216, 1296
7, 49, 343, 2401
8, 64, 512, 4096
9, 81, 729, 6561
10, 100, 1000, 10000
```

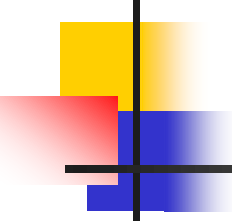
Reading File Using Pandas

```
In [18]: import pandas as pd  
nba = pd.read_csv('nba2013.csv')  
  
nba[0:10]
```

Out[18]:

	Name	Position	Height	Weight	Age in 2013
0	DeQuan Jones	Guard	80	221	23
1	Darius Miller	Guard	80	235	23
2	Trevor Ariza	Guard	80	210	28
3	James Jones	Guard	80	215	32
4	Wesley Johnson	Guard	79	215	26
5	Klay Thompson	Guard	79	205	23
6	Thabo Sefolosha	Guard	79	215	29
7	Chase Budinger	Guard	79	218	25
8	Kevin Martin	Guard	79	185	30
9	Evan Fournier	Guard	79	206	20

	A	B	C	D	E
1	Name	Position	Height	Weight	Age in 2013
2	DeQuan Jones	Guard	80	221	23
3	Darius Miller	Guard	80	235	23
4	Trevor Ariza	Guard	80	210	28
5	James Jones	Guard	80	215	32
6	Wesley Johnson	Guard	79	215	26
7	Klay Thompson	Guard	79	205	23
8	Thabo Sefolosha	Guard	79	215	29
9	Chase Budinger	Guard	79	218	25
10	Kevin Martin	Guard	79	185	30



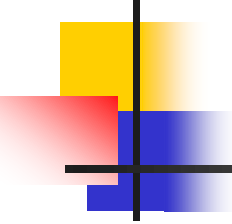
Input from the Terminal

```
In [1]: x = input()
        print(x)
        abc
        abc
```

File IO Methods



Method	Description
<u>close()</u>	Closes the file
<u>detach()</u>	Returns the separated raw stream from the buffer
<u>fileno()</u>	Returns a number that represents the stream, from the operating system's perspective
<u>flush()</u>	Flushes the internal buffer
<u>isatty()</u>	Returns whether the file stream is interactive or not
<u>read()</u>	Returns the file content
<u>readable()</u>	Returns whether the file stream can be read or not
<u>readline()</u>	Returns one line from the file
<u>readlines()</u>	Returns a list of lines from the file
<u>seek()</u>	Change the file position
<u>seekable()</u>	Returns whether the file allows us to change the file position
<u>tell()</u>	Returns the current file position
<u>truncate()</u>	Resizes the file to a specified size
<u>writable()</u>	Returns whether the file can be written to or not
<u>write()</u>	Writes the specified string to the file
<u>writelines()</u>	Writes a list of strings to the file



Summary

- Collections
 - Lists
 - Tuples
 - Dictionary
- Iteration (Loops)
 - For and While loops
 - Comparisons – If-elseif-else
- String Formatting
- File IO

Simplifying Your Code: Functions and How to Use Them

Introduction to Python Programming

Overview

In this module, you are going to learn how to write functions. The ability to write functions is a huge step up in programming ability, and programmers who learn this skill will be able to write more organized and efficient code. Your code will be easier to maintain, understand, and share with others or share between your own scripts. Similarly, you can define your own functions. These can be used anywhere in your code, so that you don't need to retype the content of that function every time you want to use it. In short, functions allow programmers to write a piece of code one time and use it everywhere, many times.

Unfortunately, many people who pick up programming only learn to write scripts. They never really learn how to write their own functions, and this often leads to very long and confusing code. Functions also help programmers stick to one of the universal laws of programming captured in the acronym DRY— that is, “Don't repeat yourself!” This principle can also be understood by its opposite term WET, which can mean “Write everything twice,” “Waste everyone's time,” or “We enjoy typing.” Thus, the goal in using Python, or any language, is to be effective, yet succinct.

Example

As we begin to discuss functions in greater detail, let's start with a simple example of a programming scenario. Remember our *if/else* statement for determining the shipping cost of a package from the previous module? This was our initial code:

```
if package_weight < 10:
    shipping_cost = 5
elif package_weight < 15:
    shipping_cost = 7
else:
    shipping_cost = 10
```

Let's imagine that the code we initially used to run our shipping company has now grown to a much more feature-rich and complex system. We now keep track of package pricing,

inventory, shipping routes, etc., and our code base is now many thousands of lines. In addition, our code needs to calculate package shipping costs at five different locations in the code.

To attempt to solve this issue, we decide to copy the code above and paste it into these five different places. However, this is not a good choice! Why not? This is because a month later we need to change the highest shipping cost to \$11 instead of \$10. Now, we need to search through our code to find all five of those places and change the cost from \$10 to \$11 in each of them. Unfortunately, we forgot that we had the code in five locations, and we only changed four of them, inadvertently creating various price structures.

On reflection, we can recognize these inconsistencies in our code and also realize that it took too much time to search through our code to make the changes. Yet, how could we have avoided this type of situation in the first place? By using a function! Use of a function in this type of scenario can give us a way to package the code, so we can simply refer to the entire code block as one unit by name.

Next, let's consider how we might go about writing functions in Python.

Writing Functions

Here is the syntax for our example of a Python function:

```
def calculate_shipping_cost(package_weight):  
    if package_weight < 10:  
        shipping_cost = 5  
    elif package_weight < 15:  
        shipping_cost = 7  
    else:  
        shipping_cost = 10  
    return shipping_cost
```

Notice that the overall syntax is similar to the *if/else* statements and *for* loops which we have previously considered, and the line at the top also ends in a colon or ':' symbol. In this syntax, the first line is called the *function signature*. The indented code below is the *content of the function*, ending with a *return line*. Let's examine each of these in more detail.

1. **Function signature:** The first line starts with '`def`' which is the keyword that lets Python know that we are *defining a function*. The next part, '`calculate_shipping_cost`', is the name of the function. This name can be anything we want as long as it starts with a letter or underscore and only contains letters, underscores, or numbers. We then indicate how many inputs the function will take.

This is determined by the number of argument names that are listed between the parentheses. In the example above, our function only takes one argument, and this can be seen because there is only one name, 'package_weight', in between the parentheses. Thus, 'package_weight' is the name of this argument *inside* of the function. Finally, the first line ends in a colon or ':' symbol.

2. **Function content:** Below the function signature is the *content of the function*. These lines of codes are indented and run every time the function is used (i.e., executed).
3. **Return line:** The last line is *the return line*. We start this line with the 'return' keyword, and then write the variable that we want to return, which in this case is 'shipping_cost'.

Defining the function like this allows us to reuse it to process various package weights. Now, let's consider an example.

Example

In the code below, we define our function and then use the function three times for three different package weights (5, 21, 14).

```
def calculate_shipping_cost(package_weight):  
    if package_weight < 10:  
        shipping_cost = 5  
    elif package_weight < 15:  
        shipping_cost = 7  
    else:  
        shipping_cost = 10  
    return shipping_cost
```

```
ship_cost_1 = calculate_shipping_cost(5)  
ship_cost_2 = calculate_shipping_cost(21)  
ship_cost_3 = calculate_shipping_cost(14)  
print(ship_cost_1)  
print(ship_cost_2)  
print(ship_cost_3)
```

Note: Only the indented lines are part of the function, and the function ends at the last indented line, which is also the return line. The lines that follow are not indented.

Every time we use the function, as we do in the line "ship_cost_1 = calculate_shipping_cost(5)", all of the code inside the function runs with whatever

value is passed to it. So, in this case we pass to the function the value of 5. This value then runs the code inside the function with the 'package_weight' argument equal to the value 5. The function calculates the shipping cost and returns that value from the function. This is the output of the function, and this is the value that is assigned to 'ship_cost_1'

The above example is very simple. We have defined our function and then used it immediately after we have defined it. However, this is not usually the case. Usually, function definitions are at the bottom of the file and then are used throughout the code. Functions can also be defined in a separate file and imported into the script and used. The structure of a project can be even more involved than that with multiple files of code, but this consideration is beyond the scope of this course.

Built-In Functions

We have already seen several examples of built-in functions in this course, but now that we have an understanding of what functions are, maybe we can appreciate them a little more. *Built-in functions* are functions that are built into Python and can be used at any time in creating code. Thus far, we have seen the functions *type* and *print*, but a complete list of all built-in functions can be found [here](#).

Let's now consider the round function, another example of a built-in function in Python.

The Round Function

The round function is a built-in function that can be used without importing anything special. We can use the round function anytime we need to round a number, and we don't have to type out all of the lines of code that go into rounding a number. There are actually several lines of code running each time we use the function *round*. However, we don't see them because they have been packaged up into the round function. (As a side note, the underlying lines of code that define *round* are not lines of Python code. Rather the code is written in C so the function is faster.)

The round function can be used in this way:

```
my_rounded_num = round(1.7)
print(my_rounded_num)
```

Other Examples

Overall, using built-in functions is very simple, and here are some other examples:

```
my_name = "William"
```

```
print(my_name)
a_number = 17.5
b_number = round(a_number)
print(a_number)
```

Now, let's wrap up this week's module with a look at what is called the standard library.

The Standard Library

The *standard library* is a library of Python modules that provide many additional functions, well beyond those that are included in the built-in functions. The standard library was developed by the Python developers and almost always comes with Python. These modules provide essential functions that many other modules are built on. You can see a list of the modules included in the standard library [here](#).

In a future lesson, we will dig into a few of these, but for now let's look at how to import a Python module from the standard library so it can be used in your code. Unlike built-in functions, which can be used anytime, functions in the standard library can only be used after the module has been imported into your code.

So, let's say we want to use the 'os' module. This is very simple! We can import it like so:

```
import os
```

Important statements almost always go at the very top of your code, and in a Jupyter notebook they should be in the first cell. You must run this cell to import the module. Once you run the cell, that module will be available anywhere in your notebook.

Now, try the following in the first cell in a notebook:

```
import os
os.getpid()
```

This imports the *os* module and then uses the *getpid* function that is inside that module. This just returns the process ID of the Python process (in this case, the Jupyter Notebook).

We don't need to be an expert in this function, and this is only an example of how to use a function from the *os* module. You can learn more about the *os* module [here](#).

We can also import just the function names, and not the module name, in this way:

```
from os import getpid()
getpid()
```

It may seem like this code is more efficient since it looks like we are only importing one function and not the entire module, but this is not correct. In both cases, the entire module is being read into memory. The only difference is that in one example you would write `'os.getpid()'` to use the *getpid* function, and in the other example you need to write `'getpid()'`.

As mentioned, in a future lesson we will look at a few more examples of modules included in the standard library.

Conclusion

In this week's lesson, we've considered the importance of learning how to write your own functions. Learning this important programming skill will lead to more organized and efficient code which will also be easier to understand, maintain, and share. Python's built-in functions and the standard library provide a good foundation for using functions.